

樹搜尋原型可行性報告(Phase C-3)

產生方式:數字逐字取自 `tree_search/harness.py`、三份 `competitions/playground-series-{s3e9,s3e14,s3e5}/experiments_tree.json`,以及對應 `STATUS.md` 的樹搜尋附錄;抽取腳本 `docs/scripts/build_tree_facts.py` → `docs/tree_facts.json`。計畫書脈絡:ERA(Aygün et al. 2026, Nature)——用候選樹搜尋取代線性單路徑迭代,核心機制為節點評分、選擇規則、plateau 偵測與回溯、以及「想法注入」持續擴充候選集。本報告是計畫書 Stage 4(週 6–7)的先遣驗證,回答一個問題:在本專案既有的三場 **playground-series** 競賽上,樹搜尋這個原型值不值得投入 Stage 4 的完整實作?

1. 目的與設計

設計目標

`tree_search/harness.py` 是一個與競賽無關的通用引擎;每場競賽只需提供一個 `evaluate(config) -> score` 函式(`tree_search/eval_s3e9.py` / `eval_s3e14.py` / `eval_s3e5.py`)。設計選擇如下:

- **node = 一個完整、已評分(或已失敗)的解:** `{id, parent_id, mutation, config, score, status, wall_s}`。不是「部分想法」或「一個超參數變動」的抽象節點,而是一個可獨立重現、有 CV 分數的完整 pipeline 設定——這是刻意的簡化,換取「隨時可比較任兩個節點」的簡單性。
- **選擇規則(selection rule):**任一時刻恰有一條「活躍 lineage」(root 的某個第一代子節點所代表的候選子樹)。`select_next_parent()` 永遠在活躍 lineage 內,挑選「分數最好、且尚未達到 `MAX_CHILDREN_PER_NODE (=3)`個子節點」的節點來擴展。
- **plateau/backtrack:**每當活躍 lineage 新增一個子節點,若其分數未能刷新「新增前的全域最佳分數」,該 lineage 的未改進計數 +1;連續達到 `PLATEAU_STREAK (=3)`次,該 lineage 標記為「已 plateau」並從候選中剔除, `select_next_parent()` 改選次佳、尚未 plateau 的 lineage(即回溯)。兩個額外規則防止搜尋卡死:(1) 一條 lineage 若耗盡擴展預算(每個節點都已有 3 個子節點)也視為 plateau;(2) 若所有 lineage 同時 plateau,清空 plateau 旗標一次(reopen),讓搜尋能繼續花完節點預算。
- **solo + blend 雙節點空間(Phase C-2b 起):** `eval_s3e14.py` / `eval_s3e5.py` 支援兩種節點型別——`solo` (訓練一個模型,OOF/test 預測快取到磁碟)與 `blend` (對已快取的成員 OOF 做權重搜尋,不重訓)。這是修正 s3e9 教訓的關鍵設計:單模型節點空間永遠碰不到集成解法能到達的分數。

- **OOF 快取:** `harness.next_id()` (公開包裝)讓 solo 節點在 `add_node()` 真正指派 id 之前就能預先知道自已的 id,把 OOF/test 矩陣存成 `tree_search/cache_<comp>/solo_<node_id>.npz`, 後續 blend 節點直接讀取、不必重新訓練——這是 blend 節點能做到「近乎免費評分」的唯一原因。

與 ERA 的對應與簡化

ERA(Aygün et al. 2026)理念	本原型的對應	簡化了什麼
候選樹(candidate tree)取代單路徑迭代	<code>harness.py</code> 的 <code>node/lineage/select_next_parent</code> 機制,完整實作	無
Plateau 偵測與回溯	<code>PLATEAU_STREAK (=3)+ lineage 排除 + reopen-once</code> 保底	固定閾值,未依指標特性(離散 vs 連續)動態調整——見第 4、6 節
想法注入(idea injection,持續產生新候選方向)	未實作: 每條 lineage 的變異佇列(mutation queue)是驅動腳本(<code>run_s3e9.py</code> 等)裡手寫的固定清單,佇列用盡後退化為「換 random seed」的填充策略	ERA 的第二支柱(見第 5 節已知缺口)
多機並行搜尋候選子樹	單機序列: <code>harness.py</code> 的 docstring 明載「no concurrent add_node calls」, <code>add_node / save</code> 假設嚴格單一寫入者	無平行化,樹的深度/廣度受限於單一序列執行時間
變異提案者 (mutation proposer) 為系統的一部分、可自我演化	固定為 agent(Claude) 在撰寫驅動腳本當下手動設計的規則式佇列,執行期間不再產生新想法,只依評分排序既有佇列	無運行時想法生成,退化為「窮舉一份預先寫好的候選清單,由樹搜尋機制決定探索順序」

簡言之:本原型完整驗證了 ERA 的**搜尋機制**(選擇/plateau/回溯/斷點續傳),但尚未觸及 ERA 真正被引用的差異化能力——**想法注入**。這是解讀第 2-4 節結果時的關鍵前提:任何「樹搜尋沒有贏」的案例,都可能是「候選集本身不夠豐富」造成,而非搜尋機制本身的缺陷。

2. 三場結果總表

comp	metric	線性最佳	樹最佳	勝/平/負	節點數	超車節點數 (evaluation-to-beat)	回溯次數	牆鐘時間
s3e9	RMSE(min)	12.07003	12.07459	負	20	從未超車(僅重新發現既有單模最	4	125.4s

comp	metric	線性最佳	樹最佳	勝/ 平/ 負	節 點 數	超車節點數 (evaluation-to-beat)	回溯次 數	牆鐘時 間
						佳)		
s3e14	MAE(min)	340.59891	340.52635	勝	20	9(node #8, 340.59485 已超車;最終於 node #11 拉開 0.07256)	2	593.4s
s3e5	QWK(max)	0.56769	0.56766	平	40	從未超車,差距僅 0.00003	11(含 1 次 reopen)	696.9s

3. 逐場分析

s3e9(單模型空間困於噪音上限)

Root 為 s3e9 既有的正則化單一 LGB(RMSE 12.11061,5.2s);5 條第一代 lineage(CAT/XGB/FEAT/ROBUST/REG)全部是**單模型**變異。20 個節點全部評估成功(0 失敗),每節點成本 2.0–17.0s,總牆鐘 125.4s——比 20–60s/節點的設計目標更便宜。全樹最佳分數是 CatBoost 種子節點本身(12.07459),CAT 自己的 3 個子節點(加正則、縮容量、加 bagging_temperature)全部打平或輸給父節點;4 次回溯(CAT→REG→FEAT→XGB 依序 plateau)都沒能找到更好的方向。

教訓:此分數在單模型空間裡已是局部最優——線性迭代真正的最佳分數(12.07003)來自 7-way seed-bagged blend,樹搜尋因為節點空間限定在單模型,結構上就到不了那裡(樹最佳 12.07459 比線性最佳差 0.00456)。這不是搜尋機制失敗,是候選空間設計失敗——直接促成 s3e14 加入 blend 節點型別的設計決策。

s3e14(勝:blend 節點 <1s、fold-exact 驗證)

Root 沿用既有調校 LGB(342.02154,52.1s)。7 條第一代 lineage:6 條 solo(CAT/XGB/LGBTUNED/SEEDBAG/FEAT/REG,用來填充 OOF 快取池)+ 1 條 BLEND(對快取成員做 Dirichlet 或 grid-simplex 權重搜尋)。20 個節點、0 失敗,12 個 solo(19.8–81.4s)+ 8 個 blend(0.5–0.8s)。

BLEND 種子節點(3-way:root+XGB+CAT)一評分就是全樹最佳,harness 隨即持續拉著這條 lineage 擴展:加入 LGBTUNED 後在第 9 個評估(node #8, 340.59485)就**超車**了線性迭代耗費 7 次實驗(每次都是完整多模型 pipeline)才找到的 340.59891;之後加入 SEEDBAG(node #9, 340.54904)、加入 FEAT(node #11, **340.52635**,全樹最終最佳)。node #10(在同一組 5 個成員上把權重搜尋法從

dirichlet 換成 grid_simplex)重現了線性迭代 exp #7 的**逐位精確**結果(raw 340.65207 / snap 340.59891),確認兩套流程在同一組 5-fold 上完全可比——這是本原型唯一一次做到「fold-exact」交叉驗證。2 次回溯都發生在最佳分數已找到之後(node #14 BLEND lineage plateau、node #17 LGBTUNED lineage plateau),說明 plateau 規則正確地把剩餘預算轉去探索新的 solo 多樣性來源,而非在已飽和的 blend lineage 上空轉。

贏的原因:winning node(#11)的第 6 個成員——把 21 特徵集再砍 3 個溫度欄位的 LGB(solo 本身 342.02283,單模並無進步)——線性迭代從未試過這個成員組合;它的價值純粹是 blend 多樣性,只有節點空間能展開到 ensemble 才碰得到。

s3e5(平:成熟 blend 空間只被重新發現;離散指標階梯面)

Root 為 Optuna 直接以 post-rounder QWK 為目標調校出的 LGB(0.56244,1.4s)——線性迭代自己已經用最高槓桿的配方把單模天花板頂到位。8 條第一代 lineage(ROOTQ + 6 solo:CAT/XGB/LGBNUDGE/FEAT/CATORIG/LGBORIG + BLEND)。40 個節點、0 失敗:30 solo(0.6–27.1s)+ 10 blend(45.0–46.5s)。樹最佳是 node #11(4-way blend:root + tuned-CAT + XGB + FEAT-17特徵版,權重 0.129/0.645/0.018/0.208,cutpoints [3.590, 4.609, 5.614, 6.160, 7.597]),QWK **0.56766**——本質上以 4 個成員重新發現了線性迭代 6-way champion 的同一個最適區(該 champion 有效權重其實也只集中在 2 個成員上),與線性最佳 0.56769 只差 **0.00003**,量級小到單一樣本的切點歸屬就能翻轉。

離散指標帶來的三個新現象(此為本次泛化測試的核心產出):

1. **分數面呈階梯狀,平手極常見:**40 節點中出現 4 組 5 位小數完全相同的分數(0.56725×3 、 0.56664×2 、 0.56466×2 、 0.56065×2)。連續指標(s3e9 的 RMSE、s3e14 的 MAE)幾乎不會出現這種平手。
2. **PLATEAU_STREAK=3 對離散指標實質上更嚴格:**8 條 lineage 全數在本輪 plateau(這是三場裡最徹底的一次),必須觸發 reopen-once 規則才能把預算花完;reopen 後 BLEND lineage 吃進新出爐的 solo 池成員,得 0.56725,仍未超越 node #11。
3. **「blend 便宜」的假設不成立:**每個候選權重都要做一次 Nelder-Mead 擬合 OptimizedRounder,單一 blend 節點成本升到 ~45–46s,與 solo 節點(0.6–27.1s)同量級甚至更貴——s3e14 建立的「blend 節點 <1s、近乎免費」經驗無法直接遷移到離散指標場景。

一個順帶發現的實作缺陷:同一 parent 的 blend fallback 因 parent config 不變而重複產出同一組成員(node #37/#38/#39,同分 0.56725),在同分頻繁的離散面下浪費了 plateau 額度——已記錄於第 5 節「已知缺口」。

4. 樹搜尋何時贏——三場歸納的邊界條件

1. **贏的條件:blend 組合空間存在,且尚未被線性迭代充分開採。**s3e14 是唯一的勝場——線性迭代在達到 340.59891 後就依「2-3 輪增益遞減即停」的協定收手,樹搜尋額外開採出的第 6 個 blend 成員(一個單模無進步、但提供多樣性的特徵子集)是線性迭代協定下不會再花預算去試的方向。
2. **平/負的條件:線性迭代已把該指標的可用信號榨乾。**s3e5 的線性迭代已用「Optuna 目標函式直接設為 post-rounder QWK」這個全場最高槓桿配方把單模與 blend 都推到局部最優,樹搜尋的候選集(手寫佇列)沒有新方向可探,只能重新發現同一最適區(平手,差 0.00003)。s3e9 則是節點空間設計就不對(單模型),樹搜尋連「有機會贏」的入場券都沒有。
3. **離散指標需要調整 plateau 規則:**QWK-after-rounder 這類離散化指標的分數面呈階梯狀,平手頻繁、有效「改進事件」更稀疏,固定 `PLATEAU_STREAK=3` 在此比在連續指標上更快把每條 lineage 都判定為停滯(s3e5 8/8 lineage 全數 plateau,是三場中最徹底的一次,連續指標的 s3e9/s3e14 都不需要 reopen)。
4. **「blend 便宜」是指標相關,不是普遍真理:**s3e14(MAE,線性權重搜尋)的 blend 節點成本 <1s;s3e5(QWK,權重搜尋內含 Nelder-Mead 切點擬合)的 blend 節點成本 ~45-46s,與 solo 節點同量級。任何以「blend 節點近乎免費」為前提的搜尋預算規劃,必須先確認該指標的後處理成本結構。

5. 已知缺口

- **重複子節點:**s3e5 的 blend fallback 在 parent config 不變時會對同一 parent 重複產生同一組成員(node #37/#38/#39 完全同分),在同分頻繁的離散面上浪費 plateau 額度——待修正:fallback 應檢查兄弟節點已試過的成員集合。
- **blend-rounder 成本:**離散指標(QWK)下 blend 節點成本(~45-46s)追上甚至超過部分 solo 節點,「多開 blend 節點幾乎無代價」的 s3e14 經驗不能直接套用,搜尋預算分配需要按指標重新估計。
- **無想法注入(ERA 第二支柱):**三場的每條 lineage 變異佇列都是驅動腳本裡手寫的固定清單(`CAT_QUEUE`、`XGB_QUEUE`.....),佇列用盡即退化為「換 seed」填充,搜尋期間不會依評分歷史動態產生新的候選方向。目前的「樹搜尋」本質上是「用回溯機制決定執行順序的一份預寫候選清單」,而非 ERA 描述的持續擴充候選集。
- **單場單樹,無跨場遷移:**三棵樹彼此獨立——沒有機制把 s3e14 學到的「blend 節點型別+加入不取代成員」直接注入下一場的初始候選集;每場都是重新手寫種子與佇列。
`knowledge/experience.md` (線性迭代的跨場經驗庫)目前只被拿來當作驅動腳本註解裡的「不要重試」提示,並未真正驅動候選生成。

6. Stage 4 建議

1. 預設節點空間應從一開始就含 **ensemble/blend** 節點型別,不要等單模型空間 plateau 後才加 (s3e9 教訓)。Stage 4 的 harness 應把 `kind: solo|blend` 設為所有新競賽的預設 schema,而非 Phase C-2b 才追加的擴充。
2. **PLATEAU_STREAK** 應依指標離散度調整,而非全域固定為 3:對 QWK/離散化指標一類的競賽,建議放寬到 4-5,或改用「與次佳分數的相對差距是否小於 tie 容忍度」判斷改進,避免像 s3e5 那樣 8/8 lineage 在同一輪內全數 plateau、被迫依賴 reopen-once 兜底。
3. 把 **knowledge/experience.md** 接上變異提案者(**mutation prior**)、銜接 ERA 的想法注入支柱:目前手寫佇列裡引用的「已證實方向」(regularize-over-capacity、seed-bagging、blend 加入不取代、snap-to-grid/OptimizedRounder)應該結構化成可程式化查詢的候選生成規則庫,讓下一場競賽的第一代 lineage 種子與後續變異佇列直接從經驗庫生成,而不是每場重新手寫——這是在不引入 LLM 運行時想法生成的前提下,最低成本地部分實現 ERA 第二支柱的做法。
4. **下一步實驗清單**:(a) 修正 s3e5 的 blend fallback 重複子節點缺陷(按已試成員集合去重);(b) 在一場資料量更大、線性迭代尚未把 blend 空間充分開採的競賽上重跑,驗證 s3e14 的勝利是否可重現而非個案;(c) 嘗試以 agent 在搜尋期間即時讀取評分歷史動態生成變異(取代固定佇列),對照本原型量化「想法注入」實際帶來多少額外增益;(d) 用經驗庫規則自動生成初代 lineage 種子,測試「零手寫佇列」的樹搜尋是否仍能達到本報告的勝場水準。

7. 重現指令

```
cd /home/tjyen/ai_agents/kaggle

# s3e9 - 單模型節點空間(Phase C-2a)
uv run python3 tree_search/run_s3e9.py

# s3e14 - solo + blend 節點空間(Phase C-2b)
uv run python3 tree_search/run_s3e14.py

# s3e5 - 離散化指標(QWK-after-rounder)泛化測試(Phase C-2c)
uv run python3 tree_search/run_s3e5.py
```

三支腳本皆可中斷/續跑(每個節點寫入後立即以 `temp-file + os.replace` 原子寫入 `experiments_tree.json`)。若要從零開始重跑,先刪除對應的 `competitions/playground-series-<comp>/experiments_tree.json` 與 `tree_search/cache_<comp>/` (皆為可重新產生、已 gitignore 的暫存)。

事實抽取與驗證:

```
uv run python3 docs/scripts/build_tree_facts.py
uv run python3 .claude/skills/kaggle-report/assets/verify_report.py docs/tree_search
bash .claude/skills/kaggle-report/assets/md2pdf.sh docs/tree_search_prototype.md
```

